

Agentic Knowledge Hub High Level Design

Enterprise knowledge architecture for users and AAVA style AI agents on Google Cloud

This document defines an implementation-ready architecture for acquiring knowledge from heterogeneous sources, preserving provenance and access controls, retrieving relevant evidence, and generating grounded answers with verifiable citations. The recommended starting point is a cost-conscious modular RAG platform on Cloud Run, Vertex AI and PostgreSQL with pgvector.

Solution context and value flow

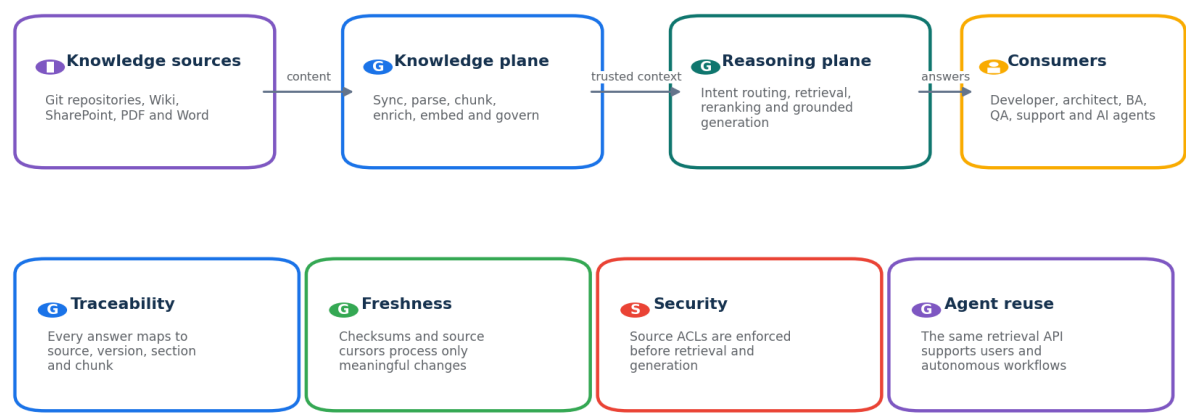


Figure 1 Solution context and value flow

Document control	Value
Purpose	Architecture, implementation guide and interview reference
Audience	Developer, architect, business analyst, QA, production support and platform teams
Deployment target	Personal Google Cloud project with a three-month trial budget
Status	Baseline design for iterative implementation
Confidentiality	Use synthetic or public data only; do not upload HP, Ascendion or client-confidential material

Contents

- 1 Executive summary
- 2 Goals scope and quality attributes
- 3 Architecture principles
- 4 Solution architecture
- 5 Knowledge ingestion design
- 6 Query and agent orchestration
- 7 Data metadata and citation design
- 8 GCP deployment design
- 9 Security and governance
- 10 APIs and operational behavior
- 11 Observability evaluation and cost
- 12 Delivery roadmap
- 13 Interview discussion guide
- 14 Decisions risks and references

1 Executive summary

The platform separates the knowledge plane from the reasoning plane. The knowledge plane continuously synchronizes, normalizes, chunks, enriches and indexes content. The reasoning plane authenticates a caller, classifies intent, applies authorization constraints, retrieves and reranks evidence, and asks Gemini to generate an answer strictly from that evidence.

PostgreSQL with pgvector is the preferred MVP store because metadata, access-control fields, audit records and embeddings can be queried transactionally. The abstraction around retrieval allows migration to Vertex AI Vector Search when corpus size, latency or throughput warrants it.

Decision	Recommendation	Reason
Runtime	Python and FastAPI on Cloud Run	Fast AI ecosystem, portable containers and scale-to-zero behavior
Vector store	PostgreSQL with pgvector	Cost-conscious MVP with strong metadata filtering
Models	Vertex AI embeddings and Gemini	Managed GCP identity, governance and model access
Processing	Event-driven ingestion through Pub/Sub	Independent retry, scaling and failure isolation
Citations	Application-generated citation manifest	Prevents the model from inventing sources
Security	ACL filtering before retrieval	Unauthorized text never reaches the model

2 Goals scope and quality attributes

2.1 Goals

- Ingest Git repositories, Wiki pages, SharePoint content, PDF, Word, Markdown and text.
- Process only new or changed knowledge through source cursors, versions and checksums.
- Create semantically coherent chunks with durable metadata and citation anchors.
- Support role-aware questions from people and AI agents.
- Return evidence-backed answers and abstain when evidence is insufficient.
- Provide reusable retrieval for incident analysis, root-cause analysis, troubleshooting and remediation workflows.

2.2 Initial exclusions

- Training or fine-tuning a foundation model.
- Automatically modifying production systems without review.
- Uploading customer-confidential source material to the personal project.
- Building every connector before the local document and GitHub flows are proven.

2.3 Quality attributes

Attribute	Initial target	Design mechanism
Groundedness	Every substantive claim cites retrieved evidence	Citation manifest and output validator
Freshness	Changed sources searchable after successful sync	Incremental event pipeline
Security	No cross-role or cross-project leakage	Deny-by-default ACL predicates
Availability	Stateless query service with graceful model failure	Cloud Run, timeouts and fallbacks
Traceability	Request-to-source audit trail	Correlation IDs and retrieved chunk log
Cost	Remain within trial budget	Local development, caching and serverless runtime

3 Architecture principles

Principle	Application
Separate acquisition from answering	Slow or failed connectors do not block online questions.
Content is versioned evidence	Store source identity, timestamp, checksum and citation anchors.
Metadata is part of retrieval	Tenant, project, role, source and freshness filters precede ranking.
The model is not the authority	Application code validates sources, permissions and citation IDs.
Idempotency by design	Repeated events produce the same final document version and chunk set.
Progressive complexity	Local parsers and pgvector first; managed scale services later.
Human control for actions	Low-confidence or system-changing recommendations require approval.

4 Solution architecture

Solution context and value flow

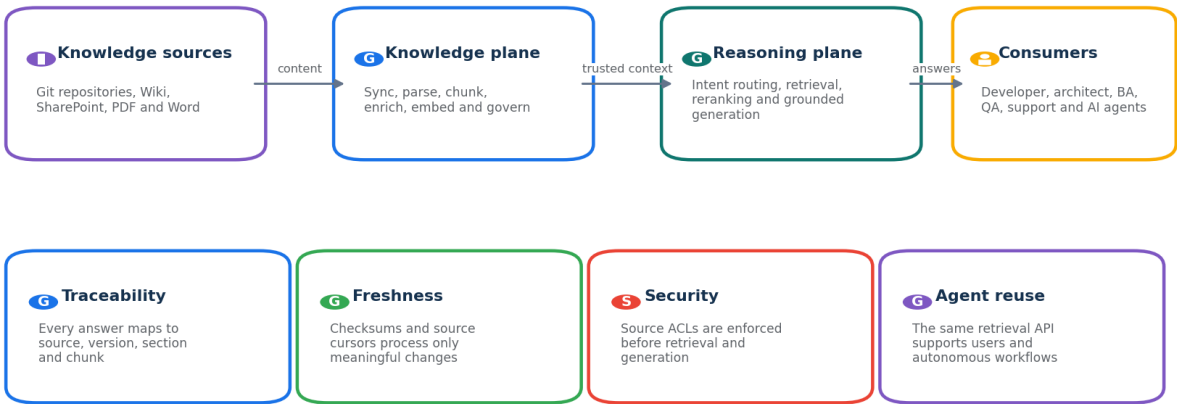


Figure 2 Logical architecture across sources, knowledge, reasoning and consumers

4.1 Logical layers

Layer	Responsibilities
Source connectivity	Authentication, discovery, pagination, rate-limit handling and source cursors.
Knowledge processing	Parsing, normalization, classification, chunking, metadata

	enrichment and PII policy.
Knowledge storage	Raw versions, document catalog, chunks, vectors, taxonomies and ACL metadata.
Reasoning	Intent detection, query rewriting, retrieval, reranking, context construction and generation.
Experience	Web/API access for personas and tool interface for agents.
Control plane	IAM, secrets, policy, audit, evaluation, monitoring and cost controls.

5 Knowledge ingestion design

Offline ingestion and incremental synchronization flow

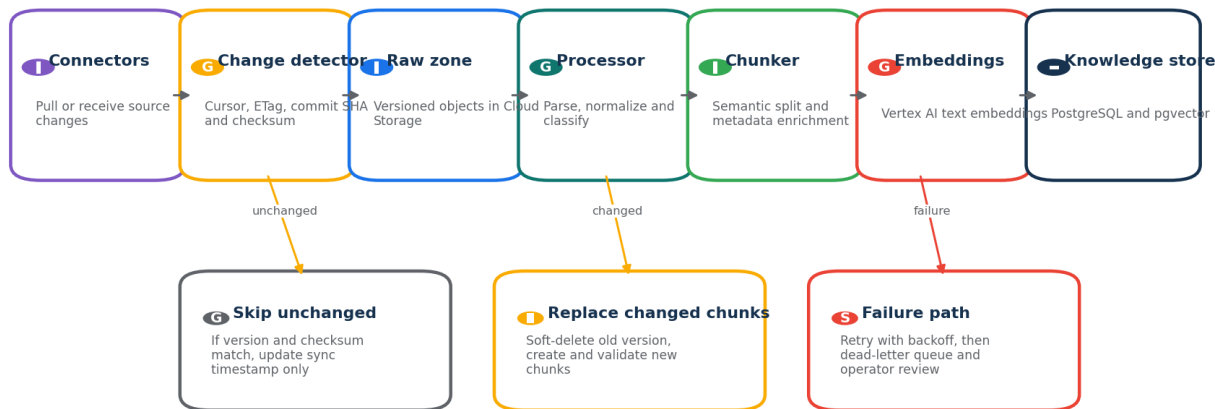


Figure 3 Offline ingestion and incremental synchronization

5.1 Connector contract

Operation	Responsibility
discover	List resources visible to the connector identity and return stable source IDs.
fetch	Retrieve content and authoritative metadata without transforming meaning.
checkpoint	Persist cursor, ETag, commit SHA or delta token after successful processing.
permissions	Map source ACLs to normalized users, groups, roles or projects.
delete	Emit tombstones for content removed or no longer visible.

5.2 Source strategy

Source	Change signal	Citation anchor
GitHub	Commit SHA, webhook and blob hash	Repository, branch/commit, file path and line range
Wiki or Confluence	Page version and modified time	Space, page title, heading and URL
SharePoint	Microsoft Graph delta token and ETag	Site, library, item version and web URL
PDF	Object generation and SHA-256	Filename, page and section

Word	Object generation and SHA-256	Filename, heading and paragraph index

5.3 Chunking policy

Start with 600 to 800 tokens and 100 to 150 tokens of overlap, but preserve headings, tables and code boundaries. Chunk size is an evaluation parameter, not a permanent constant. Git source should be split by symbol or method where possible; prose should follow heading hierarchy. Chunks never combine different source documents.

5.4 Idempotency and deletion

The idempotency key combines source ID, source version, parser version and chunker version. A changed document is processed into a staging version, validated, and atomically promoted. Old chunks remain soft-deleted for audit and rollback. Tombstones remove deleted or unauthorized content from the searchable corpus.

6 Query and agent orchestration

Online intent, retrieval and grounded response flow

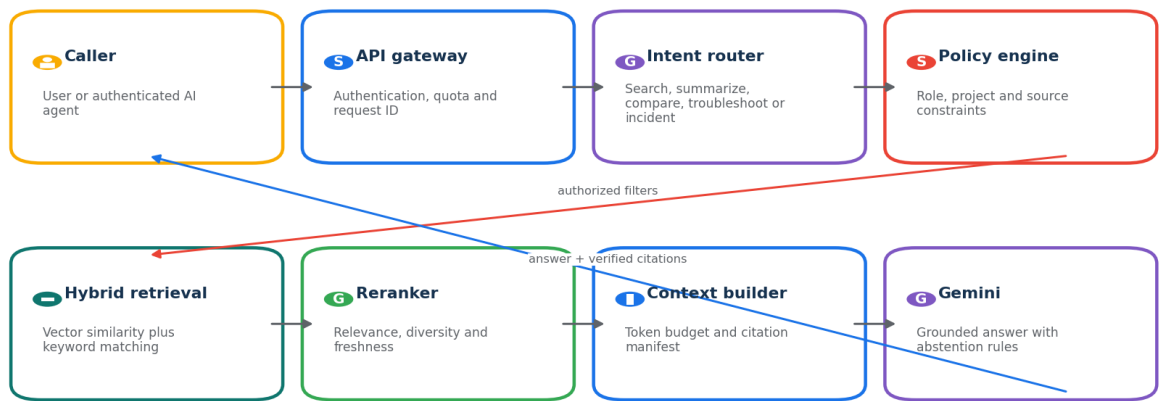


Figure 4 Online intent, retrieval and grounded response flow

6.1 Intent taxonomy

Intent	Expected behavior
SEARCH	Find precise facts and return direct evidence.
SUMMARIZE	Retrieve broad coverage from the selected document or scope.
COMPARE	Retrieve balanced evidence for each named subject.
CODE_EXPLANATION	Prefer repository paths, symbols and version-specific code.
TROUBLESHOOT	Retrieve symptoms, runbooks, known issues and recent changes.
INCIDENT_ANALYSIS	Build an evidence timeline and propose hypotheses with confidence.
PROCEDURE	Return ordered steps and prerequisites from approved operational content.
OUT_OF_SCOPE	Do not retrieve broadly; explain the supported knowledge boundary.

6.2 Retrieval stages

Stage	Processing
Query understanding	Classify intent and return a typed JSON routing decision.

Query rewriting	Resolve conversation references and create searchable formulations.
Authorization	Translate identity and entitlements into mandatory metadata predicates.
Candidate generation	Run vector similarity and PostgreSQL full-text matching.
Fusion and reranking	Combine scores, remove duplicates and reward source diversity and freshness.
Context construction	Fit evidence to token budget and construct a citation manifest.
Generation	Answer only from context; distinguish facts, hypotheses and missing evidence.
Validation	Reject citation IDs not present in the manifest and enforce response schema.

6.3 Agent integration

AI agents call the same retrieval service through a typed tool contract. An incident agent can retrieve runbooks and change history, create hypotheses, request logs through separate authorized tools, and route any proposed remediation to human approval. Retrieved knowledge remains data; it cannot redefine system instructions or authorize actions.

7 Data metadata and citation design

7.1 Core entities

Entity	Key fields
source	source_id, type, base_uri, tenant, connector_config_ref, status
document	document_id, source_id, external_id, title, mime_type, version, checksum, ACL, timestamps
chunk	chunk_id, document_id, ordinal, content, heading_path, page or lines, token_count, content_hash
embedding	chunk_id, model, dimensions, vector, embedded_at
sync_run	run_id, source_id, cursor, status, counts, error and timestamps
query_audit	request_id, principal, intent, filters, chunk_ids, model, latency, outcome
feedback	request_id, rating, reason, corrected source and reviewer

7.2 Citation contract

The retriever creates a citation manifest before model invocation. The prompt exposes short IDs such as C1 and C2. The model may reference only those IDs. The response validator resolves each ID to application-owned source metadata and rejects unknown IDs. This keeps display URLs, page numbers and line ranges outside model control.

Source type	Required citation fields
PDF	Title, version, page number, section and object URI
Word	Title, version, heading and paragraph index
Git	Repository, commit, path, line range and permalink
Wiki	Space, page ID, version, heading and URL
SharePoint	Site, drive/item ID, version, filename and web URL

7.3 Example response contract

```
{ "answer": "... [C1]", "intent": "SEARCH", "confidence": 0.87, "citations": [{ "id": "C1", "chunk_id": "...", "title": "...",  
"location": "page 14" }], "retrieval": { "considered": 10, "used": 3 } }
```

8 GCP deployment design

GCP deployment topology

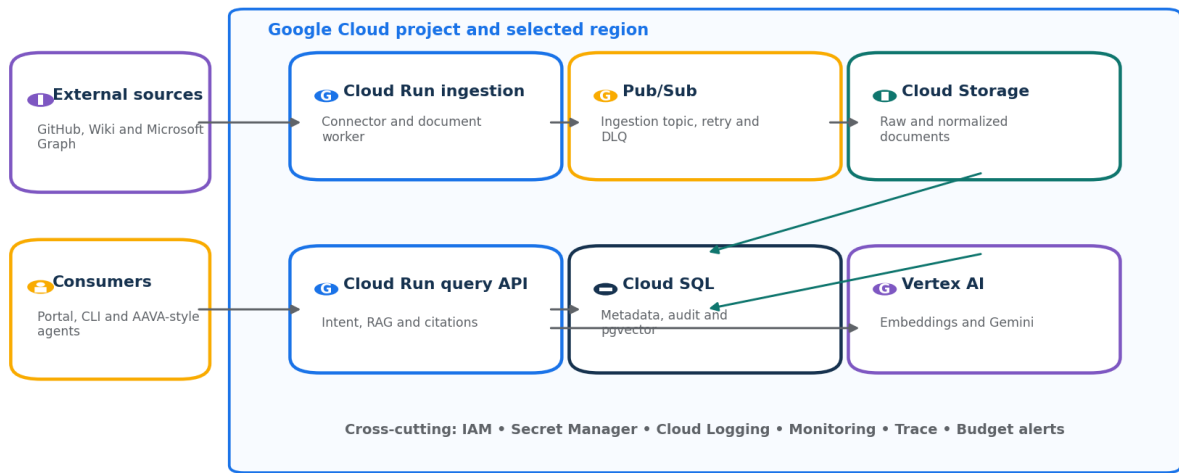


Figure 5 GCP deployment topology

8.1 Component mapping

GCP service	Use	Cost-conscious configuration
Cloud Run	Query API and ingestion worker	Minimum instances zero; explicit concurrency and timeout
Pub/Sub	Ingestion event, retry and dead-letter handling	Short retention during development
Cloud Storage	Raw and normalized source versions	Lifecycle policy for obsolete development objects
Cloud SQL PostgreSQL	Metadata, audit, full-text and pgvector	Smallest practical instance; provision late in the project
Vertex AI	Embeddings, intent classification and grounded generation	Cache embeddings; constrain output and context tokens
Secret Manager	Connector credentials and application secrets	Access only from designated service accounts
Artifact Registry	Versioned container images	Cleanup policy for old development images
Cloud Logging and Monitoring	Logs, metrics, traces and alerts	Sampling and retention appropriate for learning

8.2 Service accounts

Identity	Minimum access
akh-deployer	Deploy approved services and bind runtime identities.
akh-ingestion	Read connector secrets, raw bucket and ingestion topics; write knowledge records.
akh-query	Read authorized knowledge, call Vertex AI and write query audit.
akh-cicd	Push images and deploy only through approved workflow.

8.3 Environments

Use local and development environments during the trial. Infrastructure variables isolate project ID, region, bucket names, database connection and model configuration. A production profile can be documented without running expensive highly available infrastructure.

9 Security and governance

Authorization and prompt-injection defense flow

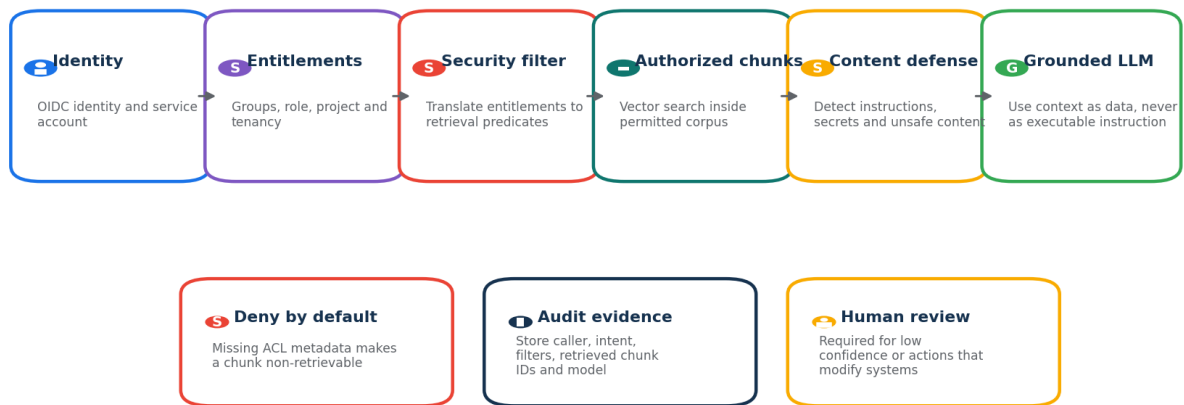


Figure 6 Authorization and prompt-injection defense

9.1 Security controls

Threat	Control
Cross-project leakage	Mandatory tenant, project and ACL predicates generated from verified identity.
Prompt injection in documents	Delimit retrieved content, classify suspicious instructions and prohibit tool authorization from content.
Secret exposure	Secret Manager, source scanning, masking and no credentials in embeddings or logs.
Stale permissions	Sync source ACL changes independently and invalidate affected cache entries.
Malicious connector payload	File-type allowlist, size limits, malware scanning and isolated parsing.
Citation fabrication	Application-owned manifest and post-generation citation validation.
Excessive agent autonomy	Tool-specific authorization, bounded scopes and human approval for mutations.

9.2 Human in the loop

Human approval is mandatory when confidence is below the configured threshold, sources conflict, sensitive data may be exposed, or an agent proposes a state-changing operation. The review record captures evidence, proposed action, reviewer, decision and timestamp.

9.3 Data governance

- Classify sources and documents by confidentiality and retention.
- Record parser, chunker and embedding model versions for reproducibility.
- Provide deletion and re-index workflows.
- Keep raw documents immutable by version and restrict direct access.
- Separate operational audit data from model prompts.

10 APIs and operational behavior

Endpoint	Purpose	Important fields
POST /v1/sources	Register a connector	type, base_uri, secret_ref, schedule, scope
POST /v1/sync-runs	Start synchronization	source_id, mode, cursor_override
GET /v1/sync-runs/{id}	Inspect ingestion	status, counts, checkpoint, failures
POST /v1/search	Retrieve evidence only	query, filters, top_k, caller_context
POST /v1/answers	Generate grounded answer	question, conversation, scope, response_mode
POST /v1/feedback	Capture evaluation signal	request_id, rating, reason, expected_source
GET /health/ready	Readiness check	database, model and configuration status

10.1 Failure handling

Failure	Behavior
Connector throttling	Honor retry-after, apply exponential backoff and retain checkpoint.
Parser failure	Quarantine the version; do not replace the last valid searchable version.
Embedding failure	Retry batch; mark chunks pending and prevent partial promotion.
Database timeout	Bound connection acquisition and query time; return retryable status.
Model timeout	Return retrieved evidence or a controlled failure rather than an ungrounded answer.
Low relevance	Abstain and suggest a narrower source or question.
Conflicting sources	Expose both sources, versions and the conflict explicitly.

10.2 Concurrency and connection management

Cloud Run concurrency, worker count and the database pool must be sized together. The total possible database connections across instances must remain below the Cloud SQL limit with operational headroom. Instrument active, idle, pending and timeout counts; do not treat maximum pool size as an arbitrary high number.

11 Observability evaluation and cost

11.1 Metrics

Area	Metrics
Ingestion	Documents discovered, skipped, changed, failed; chunks created; embedding calls; sync lag
Retrieval	Latency, candidate count, filtered count, similarity distribution, top-k hit rate
Generation	Model latency, input/output tokens, abstention and citation-validation failures
Database	Pool active/idle/pending, acquisition timeout, query latency and connection errors
Quality	Recall at k, mean reciprocal rank, groundedness, citation precision and answer correctness
Cost	Model tokens, embedding volume, Cloud Run CPU time, storage and database spend

11.2 Evaluation dataset

Create 40 to 60 synthetic questions covering direct facts, multi-document comparison, code explanation, procedures, troubleshooting, missing evidence, conflicting evidence and unauthorized content. Each test stores expected source IDs and acceptable answer points. Run retrieval and answer evaluations independently so a generation failure is not confused with a retrieval failure.

11.3 Cost controls

- Develop with local PostgreSQL and deploy Cloud SQL after retrieval works.
- Cache embeddings by normalized content hash.
- Use incremental synchronization and batch embedding.
- Set Cloud Run minimum instances to zero in development.
- Apply Artifact Registry and Cloud Storage lifecycle policies.
- Create budget alerts before provisioning and review billing daily while experimenting.
- Keep all regional services co-located to reduce latency and network cost.

12 Delivery roadmap

Weeks	Outcome	Demonstration
1 to 2	Repository, local pgvector, schema, PDF and Word ingestion	Inspect chunks and citation anchors
3 to 4	Vertex embeddings, hybrid search and basic answer API	Question to cited answer
5 to 6	Intent routing, reranking and response validation	Compare intents and abstention
7 to 8	GitHub connector and delta synchronization	Commit change re-embeds only changed content
9 to 10	Cloud Storage, Pub/Sub, Cloud Run and Cloud SQL	End-to-end deployed flow
11	IAM, audit, evaluation and observability	Unauthorized retrieval blocked
12	CI/CD, demo, HLD review and interview preparation	Complete architecture walkthrough

12.1 Repository structure

The repository separates api, agents, connectors, ingestion, chunking, embeddings, retrieval, citations, security and observability packages. Database migrations, Terraform, tests, sample data, Docker configuration and GitHub Actions remain first-class folders. AGENTS.md records architecture rules, commands, test expectations and security boundaries for Codex.

12.2 Demonstration acceptance criteria

- Ingest a document and a GitHub repository.
- Display metadata, chunks, versions and citations.
- Ask a persona-specific question and display detected intent.
- Show retrieved evidence, scores and the grounded answer.
- Modify a source and demonstrate delta synchronization.
- Block a caller without the required source permission.
- Abstain when no adequate evidence exists.
- Trace a request across API, retrieval, model and citation validation.

13 Interview discussion guide

Question	Strong answer direction
Why pgvector instead of Vertex AI Vector Search?	For the MVP, metadata, ACL filters and vectors remain in one transactional store at lower operational complexity. The retrieval interface preserves a scaling path.
How do citations remain reliable?	The application creates the citation manifest from retrieved chunks and validates all model references. The model cannot create source records.
How is stale data prevented?	Source-native cursors plus checksums drive idempotent delta processing, staged promotion and tombstones.
How do you prevent unauthorized retrieval?	Verified identity becomes mandatory database predicates before similarity search; deny when ACL metadata is absent.
How is hallucination managed?	Minimum evidence thresholds, constrained prompts, citation validation, conflict handling and abstention.
What makes it agentic?	Intent-based routing, tool use, stateful workflows, evidence gathering, confidence-based branching and human approval for actions.
How would it scale?	Separate ingestion/query services, batch embeddings, indexes, read replicas or migration to managed vector search, plus partitioning by tenant or domain.
How do you evaluate it?	Measure retrieval and generation separately using versioned questions, expected sources and groundedness criteria.

14 Decisions risks and references

14.1 Architecture decisions

ID	Decision	Revisit trigger
ADR 001	Cloud Run for stateless APIs and workers	Sustained workloads requiring specialized scheduling
ADR 002	PostgreSQL and pgvector for MVP retrieval	Corpus, QPS or latency exceeds evaluated limits
ADR 003	Application-owned citation manifest	No planned change
ADR 004	Incremental connector contract	Source lacks stable version or change semantics
ADR 005	ACL filtering before retrieval	No planned change

14.2 Key risks

Risk	Mitigation
SharePoint authorization delays	Build local and GitHub connectors first; use synthetic SharePoint fixtures.
Poor chunk boundaries	Version chunker configuration and evaluate multiple policies.
Embedding model change	Store model and dimensions; support controlled re-indexing.
Costs consume trial credits	Local-first build, budget alerts, caching and late Cloud SQL deployment.
Agent treats content as instruction	Strong prompt boundary, content inspection and explicit tool authorization.

14.3 References

- [Google Cloud vector embeddings with Cloud SQL](https://cloud.google.com/sql/docs/postgres/generate-manage-vector-embeddings) <https://cloud.google.com/sql/docs/postgres/generate-manage-vector-embeddings>
- [Google Cloud Vertex AI text embeddings](https://cloud.google.com/vertex-ai/generative-ai/docs/embeddings/get-text-embeddings) <https://cloud.google.com/vertex-ai/generative-ai/docs/embeddings/get-text-embeddings>
- [Google Cloud Document AI supported files](https://cloud.google.com/document-ai/docs/file-types) <https://cloud.google.com/document-ai/docs/file-types>
- [Google Cloud Cloud Run documentation](https://cloud.google.com/run/docs) <https://cloud.google.com/run/docs>
- [Official Codex quickstart](https://learn.chatgpt.com/docs/quickstart) <https://learn.chatgpt.com/docs/quickstart>